



Práctica Obligatoria 2: Sistemas Distribuidos

Manuel Meléndez Ortega

Índice

Introducción

Arranque del sistema

- Base de datos
- Frontend y backend
- API Rest

Pantallas

- Página principal
- Pantalla de búsqueda

Gestión y Manejo de Excepciones

- Excepciones de llamadas a APIs de terceros
- Excepciones de apertura y lectura de archivos
- Excepciones de acceso a Bases de Datos
- Excepciones críticas de Indisponibilidad del Sistema

Postman errors

- Excepciones de llamadas a APIs de terceros
- Excepciones de apertura y lectura de archivos
- Excepciones de acceso a Bases de Datos
- Excepciones críticas de Indisponibilidad del Sistema

Conclusiones

Introducción

En esta práctica se busca la creación de un sistema distribuido simple con un frontend con thymeleaf con sistema de login, un backend conectado con una base de datos almacenada en docker utilizando JPA e hibernate, una API Rest hecha con flask que se conecte al backend, a la base de datos y la API de pokemon y tenga gestión de errores.

Arranque del sistema

- Base de datos

La primera vez que se quiera enlazar la bases de datos con spring-boot se debe ejecutar el comando en la carpeta basico puesto que ahí está el archivo docker-compose.yml donde se encuentran toda la información respectiva a la base de datos.

```
PS C:\Users\manum\Desktop\Práctica2SisDis\basico> docker-compose up -d
[+] up 1/1
✓ Container basico-mysqldb-1 Running 0.0s
```

Para arrancar la base de datos local simplemente se debe seleccionar la base de datos que está almacenada en docker y darle al botón de iniciar, en este caso mysqldb-1 que almacena, los roles, los usuarios y el historial de búsqueda.

Q Search		Only show running containers		Delete		▶	
☒	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
☒	basico	-	-	-	N/A	4 minutes ago	☐ ▶
☒	mysqldb-1	560aa035c6dc	mysql:8.0.31-oracle	3506:3306	N/A	4 minutes ago	☐ ▶
☒	redis-server	d2a737f24703	redis	6379:6379	N/A	4 minutes ago	☐ ▶

- Frontend y backend

Para arrancar el frontend y el backend se debe ejecutar el siguiente comando desde la carpeta básico puesto que ahí se encuentra el archivo pom.xml que guía a java para saber cuales son las librerías que necesita descargar y como arrancar:

Práctica2SisDis > basico >

Ordenar Ver

Nombre	Fecha de modificación
.idea	23/04/2026 10:44
.mvn	16/03/2026 14:59
src	16/03/2026 14:59
target	26/04/2026 11:19
.env	16/03/2026 15:16
.gitattributes	16/03/2026 14:59
.gitignore	16/03/2026 14:59
docker-compose.yml	16/03/2026 15:16
HELP.md	16/03/2026 14:59
mvnw	16/03/2026 14:59
mvnw.cmd	16/03/2026 14:59
pom.xml	24/04/2026 11:05

✓ **TERMINAL**

```
PS C:\Users\manum\Desktop\Práctica2SisDis\basico> .\mvnw spring-boot:run
```

Es necesario tener iniciada la base de datos de antemano para que el arranque no falle.

- API Rest

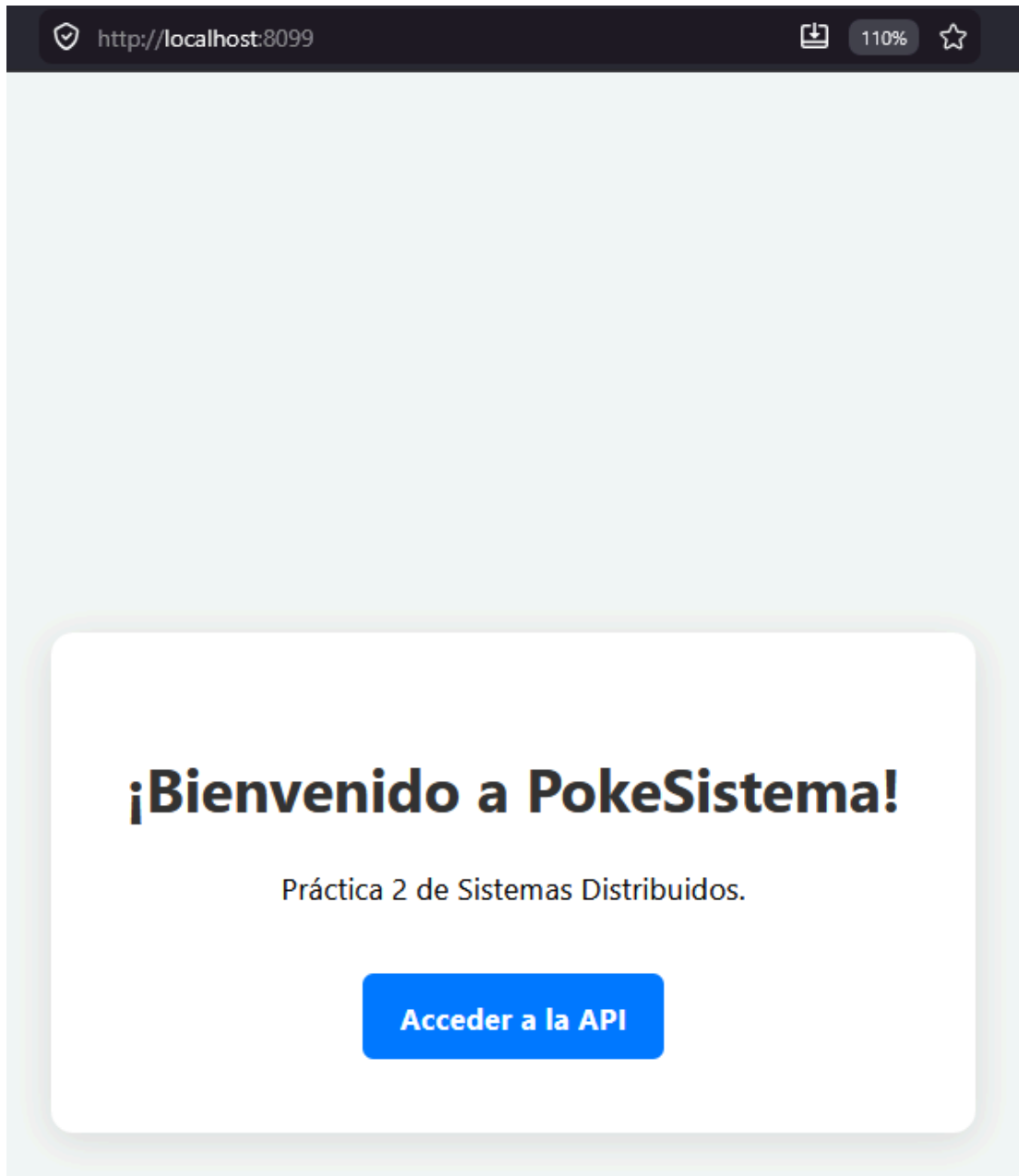
Para iniciar la API Rest solamente hace falta ejecutar el archivo API Rest.py.

api-python
 API Rest.py
 basico

Pantallas

- Página principal

Una vez todo está iniciado desde cualquier navegador se puede acceder al frontend escribiendo "<http://localhost:8099>", primeramente aparece la pantalla principal con la opción de acceder a la API.



Si se le clicca en el botón "Acceder a la API" y no has iniciado sesión anteriormente te manda a la pantalla de login.

Please sign in

Username

Password

Sign in

Dentro de la base de datos hay un usuario ya guardado con el rol de admin con el usuario "admin" y contraseña "1234" para poder acceder ya que como tal no está implementada una manera de registrarse.

Propiedades

Datos

Diagrama

Show SQL | Enter a SQL expression to filter results (use Ctrl+Space)

Grilla

	123 id	A-Z role_name	123 show_on_create
1	1	ROLE_ADMIN	1

Propiedades										Datos	Diagrama
Show SQL										Enter a SQL expression to filter results (use Ctrl+Space)	
Grilla		123 id	AZ email	fecha_ultimo_acceso	AZ nombre_usuario	AZ password	publickey	AZ username	123 user_role_id		
	1	1	admin@correo.com	[NULL]	Administrador	{noop}1234	[NULL]	admin	1		

- Pantalla de búsqueda

Una vez se ha accedido se llega a la pantalla de búsqueda donde se puede seleccionar de que pokemon queremos obtener sus datos o cerrar sesión pulsando el botón inferior "Cerrar Sesión".

Buscador de Pokémon

Introduce un nombre para consultar la **API de Python**.

Buscar Pokémon

[Cerrar Sesión](#)

Finalmente, una vez seleccionado el pokemon la API Rest consulta a la API de pokemon, se devuelven los datos y se presentan en el frontend, se han elegido solamente el nombre, el tipo, la altura y el peso del pokémon.

Datos traídos de Python

Nombre: pikachu

Tipo: electric

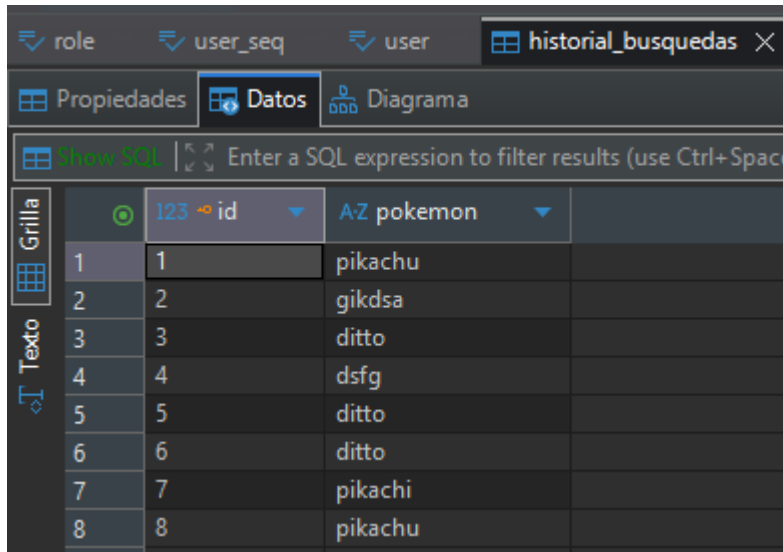
Altura: 4

Peso: 60

[Volver a buscar](#)

Una vez aquí se puede volver a realizar otra búsqueda clicando el botón “Volver a buscar”.

Independientemente de la búsqueda realizada la API Rest guarda esta información directamente en la base de datos, concretamente en la tabla "historial_búsquedas".



	123 id	A-Z pokemon	
1	1	pikachu	
2	2	gikdsa	
3	3	ditto	
4	4	dsfg	
5	5	ditto	
6	6	ditto	
7	7	pikachi	
8	8	pikachu	

Gestión y Manejo de Excepciones

Se han manejado varios tipos de excepciones:

Excepciones de llamadas a APIs de terceros

Cuando el usuario introduce un término de búsqueda inválido o un Pokémon que no existe, la API de pokemon devuelve un código HTTP 404 y se muestra el siguiente mensaje.

Datos traídos de Python

Excepción de API (404): La PokeAPI dice que 'fakemon' no existe.

[Volver a buscar](#)

Excepciones de apertura y lectura de archivos

Puesto que en el enunciado se especifica el control de excepción de apertura de archivos y esa característica no se usa en la práctica se ha codificado que

en caso de escribir "error_archivo" en el buscador te se intenta abrir un archivo inexistente y te devuelve el correspondiente error.

```
@app.route('/pokemon/<nombre>')
def get_pokemon(nombre):

    if nombre == "error_archivo":
        try:
            with open("archivo_inventado.txt", "r") as f:
                data = f.read()
        except FileNotFoundError as e:
            return jsonify({"error": f"Excepción de Archivos: No se pudo leer el archivo local. Detalle nativo: {str(e)}"}), 500
```

Datos traídos de Python

Excepción de Archivos: No se pudo leer el archivo local. Detalle nativo: [Errno 2] No such file or directory: 'archivo_inventado.txt'

[Volver a buscar](#)

Excepciones de acceso a Bases de Datos

De la misma forma que el error anterior utilizando la palabra clave de testeo "error_db", se fuerza una conexión a un puerto de base de datos erróneo (9999) mediante la librería pymysql. Esto genera un OperationalError nativo que simula una caída de la base de datos de historial. La excepción es procesada y mostrada en pantalla como un error de microservicio.

Datos traídos de Python

Excepción de Datos: Fallo al conectar con MySQL. Detalle nativo: (2003, "Can't connect to MySQL server on 'localhost' ([WinError 10061] No se puede establecer una conexión ya que el equipo de destino denegó expresamente dicha conexión)")

[Volver a buscar](#)

Excepciones críticas de Indisponibilidad del Sistema

Si la API Rest estuviera caída en el momento de intentar realizar una búsqueda se devuelve el siguiente error.

Datos traídos de Python

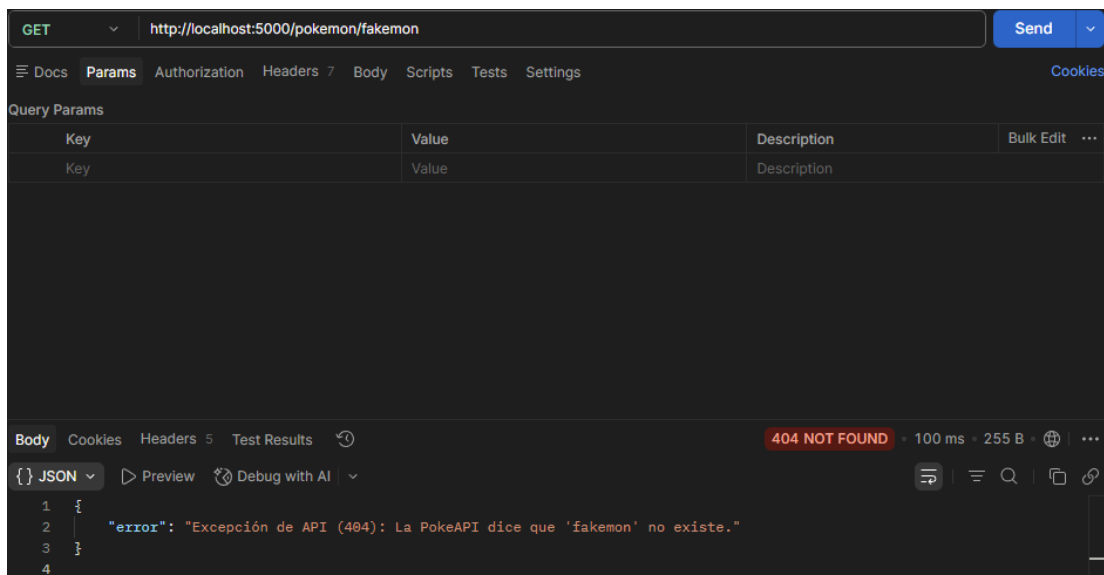
El sistema no está disponible actualmente.

[Volver a buscar](#)

Postman errors

Como se pide en el enunciado de la práctica se muestra la salida de estos mismos errores utilizando la aplicación "Postman"

Excepciones de llamadas a APIs de terceros



Excepciones de apertura y lectura de archivos

GET

http://localhost:5000/pokemon/error_archivo

Send

Docs

Params

Authorization

Headers 7

Body

Scripts

Tests

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

Cookies

Headers 5

Test Results

500 INTERNAL SERVER ERROR

4 ms

337 B

...

{ } JSON

Preview

Debug with AI

```
1 {
2   "error": "Excepción de Archivos: No se pudo leer el archivo local. Detalle nativo: [Errno 2] No such file or
3   directory: 'archivo_inventado.txt'"
4 }
```

Excepciones de acceso a Bases de Datos

GET

http://localhost:5000/pokemon/error_db

Send

Docs

Params

Authorization

Headers 7

Body

Scripts

Tests

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

Cookies

Headers 5

Test Results

500 INTERNAL SERVER ERROR

4.10 s

458 B

...

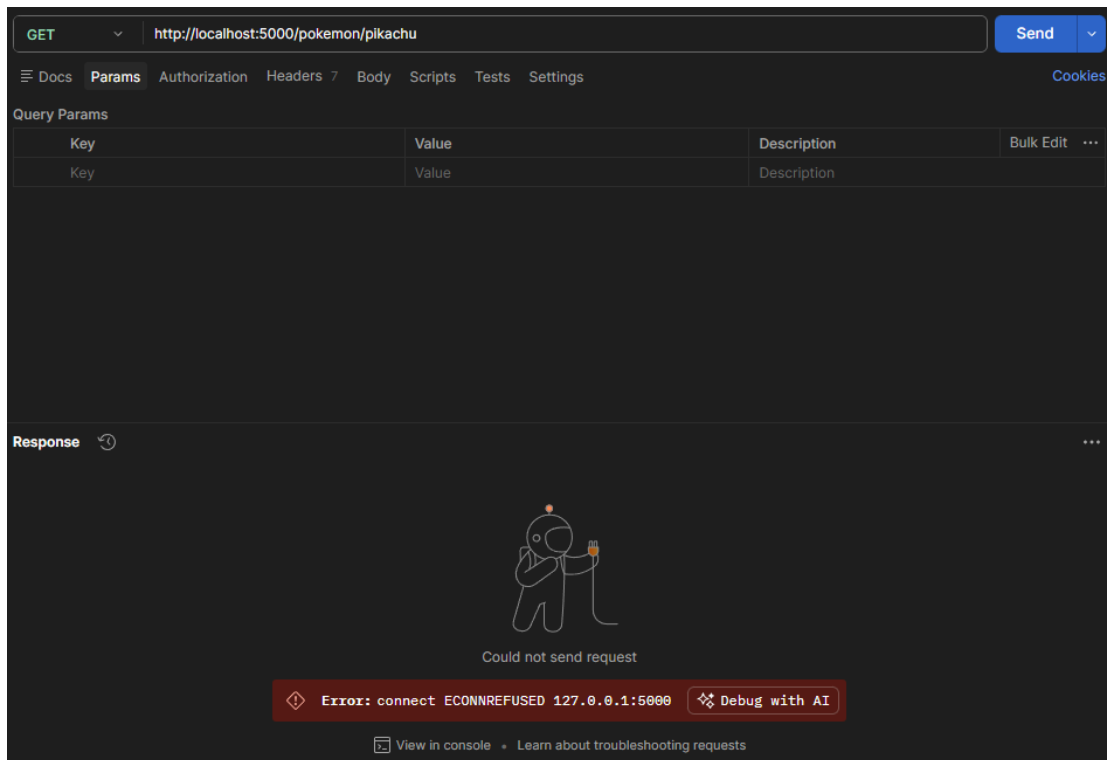
{ } JSON

Preview

Debug with AI

```
1 {
2   "error": "Excepci\u00f3n de Datos: Fallo al conectar con MySQL. Detalle nativo: (2003, \"Can't connect to MySQL
3   server on 'localhost' ([WinError 10061] No se puede establecer una conexi\u00f3n ya que el equipo de destino
4   deneg\u00f3 expresamente dicha conexi\u00f3n)\")"
```

Excepciones críticas de Indisponibilidad del Sistema



Conclusiones

En conclusión, esta práctica me ha servido para entender de forma real cómo funciona un sistema distribuido. Ver cómo se comunican un backend en Java y una API en Python, compartiendo la misma base de datos levantada en Docker, me ha ayudado a encajar todas las piezas y verle el sentido a separar las aplicaciones en diferentes partes.